

Package ‘vbayesGP’

October 5, 2026

Type Package

Title Gaussian Variational Approximation to Gaussian Process Regression

Version 1.4.1

Date 2026-10-04

Author Seongil Jo [aut, cre],
Woojoo Lee [aut]

Maintainer Seongil Jo <bstatsjo@gmail.com>

Description Implements Gaussian variational approximation to Bayesian semiparametric regression with a Gaussian process prior built on the radial basis function (RBF) kernel. Continuous outcomes are fitted directly, and binary and count outcomes by Polya-Gamma augmentation with a sparse inducing-point approximation to the process. Exposures may enter through a single kernel or through several weighted indices, giving a multiple index model; an index model with a Polya-Gamma outcome takes binary and count outcomes, its bound integrating the inducing values out analytically. The lengthscale parameters take a normal, an independent normal or a continuous shrinkage prior; under shrinkage the fit reports posterior inclusion probabilities for variable selection. Cluster-level random intercepts and slopes are available for every model, and mini-batch stochastic optimisation for the fixed-effects fits, the whole sample being used whenever random effects are present.

License GPL-2

Imports Rcpp (>= 1.0.8), fields, ggplot2, ks, dplyr, magrittr,
tibble, tidyr

LinkingTo Rcpp, RcppArmadillo, BH

Suggests bkmr, MASS, knitr, rmarkdown

VignetteBuilder knitr

Depends R (>= 3.5.0)

RoxygenNote 7.3.3

Encoding UTF-8

LazyData true

LazyLoad yes

NeedsCompilation yes

Contents

computeOverallRisk	2
computePPIPs	3
computeSingVarInt	5
computeSingVarRisk	6
extractELBO	7
extractPostSamps	8
extractRandomEffects	9
fitted.gpmim	9
fitted.gpr	10
gvaggpmim	12
gvaggpr	18
gvagpmim	22
gvagpr	26
metals	30
negbinomial	31
plot.gpmim	32
plot.gpr	33
plot.gprfitBivar	34
plot.gprfitBivarLevels	34
plot.gprfitUnivar	35
plot.risks	35
predict.gpmim	36
predict.gpr	37
predictorResponseBivar	38
predictorResponseBivarLevels	39
predictorResponseBivarPair	40
predictorResponseUnivar	42
print.gpmim	43
print.gpr	44
summary.gpmim	44
summary.gpr	45
Index	47

computeOverallRisk	<i>Calculate Overall Risk Summaries</i>
--------------------	---

Description

Compare estimated f function when all predictors are at a particular quantile to when all are at a second fixed quantile

Usage

```
computeOverallRisk(
  object,
  qs = seq(0.25, 0.75, by = 0.05),
  q.fixed = 0.5,
  nsamples = 1000,
  ...
)
```

Arguments

object	an object of class gpr obtained from gvagpr function
qs	vector of quantiles at which to calculate the overall risk summary
q.fixed	a second quantile at which to compare the estimated f function
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
...	unused argument

Value

a data frame containing the (posterior mean) estimate and posterior standard deviation of the overall risk measures

References

Bobb J (2023). `_bkmr: Bayesian Kernel Machine Regression_`. R package version 0.2.2, <<https://github.com/jenfb/bkmr>>

See Also

`gvagpr`

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
risks.overall <- computeOverallRisk(fout, qs = seq(0.25, 0.75, by = 0.05),
                                   q.fixed = 0.5, nsamples = 300)

risks.overall$summary
plot(risks.overall)
```

computePPIPs	<i>Compute pseudo posterior inclusion probabilities (PPIPs) from VGPR model or VGPMIM fits</i>
--------------	--

Description

Compute pseudo posterior inclusion probabilities (PPIPs) using Sequential-2-Means from Variational Gaussian Process Regression (VGPR) model fit or Variational Gaussian Process Multiple Index Model (VGPMIM) fit

Usage

```
computePPIPs(object, nsamples = 1000, ntuning = 10)
```

Arguments

object	an object of class gpr or gpmim
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
ntuning	the number of chosen values of the tuning parameter for variable selection

Value

a data frame of pseudo posterior inclusion probabilities. For a gpr fit there is one row per exposure. For a gpmim fit there are two levels: one row per index, from the index length-scales γ_m , and one row per exposure, from the unnormalised index weights $\psi_{ml} = \sqrt{\gamma_m} \alpha_{ml}$. The two nest, since $\gamma_m = \sum_l \psi_{ml}^2$. The normalised weights α_{ml} are not used: they lie on the unit sphere and so cannot be shrunk toward zero, which is what the sequential two-means post-processing assumes.

Rows carry the names the fit was given. Index rows take the element names of a named list Z, exposure rows the column names of the groups, and anything left unnamed falls back to Z1, Z2, ... and Z1_1, Z1_2, ..., where the trailing number counts within the index rather than across the whole of Z.

A gpmim fit adds two further columns, index and index.PPIP, which name the index an exposure belongs to and repeat that index's own probability. They are there because the two levels are read together: a weight row says which exposures carry an index only when that index is itself selected. Where an index is excluded, its weight rows are not a statement about which of its exposures survive; sequential two-means always returns two non-empty clusters, so with no signal to separate it splits noise, and some of those weights come back above one half. If no index is selected at all, that is true of every weight row. A weight above one half sitting under an index.PPIP below one half is that situation and not a selected exposure. Both levels are reported in full and neither is blanked out, since the weight level is the more accurate of the two and gating it on the index level would propagate the index level's misses.

Sequential two-means separates the larger values from the smaller ones, so it answers a comparative question and not an absolute one. Two-means returns two non-empty clusters, so of M values it selects at least one and at most $M - 1$ in every posterior draw, and the probabilities sum to between 1 and $M - 1$. The procedure can therefore report that some are active but never that all of them are, nor that none of them are. With nothing active the forced selection is spread over the candidates, so no single probability need reach one half once there are three or more of them; with everything active at least one genuine signal is called null. The bounds meet at $M = 2$, where exactly one of the two is selected in every draw and the two probabilities sum to one: for a gpmim fit with two indices the index rows say which index is the larger, not whether either matters, and a warning is given.

Whether an index or an exposure matters at all is read off summary instead, where the quantities are compared with zero rather than with each other: lambda is the amplitude of the exposure-response surface, gamma. gives each index its own length-scale, and the signed psi. rows give each exposure inside its index, all with credible intervals.

Author(s)

Seongil Jo

References

Li, H. and Pati, D. (2017). "Variable Selection Using Shrinkage Priors", Computational Statistics and Data Analysis, 107, 107-119.

See Also

gvagpr, gvaggpr, gvagpmim, gvaggpmim

computeSingVarInt

Single Variable Interaction Summaries

Description

Compare the single-predictor health risks when all of the other predictors in Z are fixed to their a specific quantile to when all of the other predictors in Z are fixed to their a second specific quantile.

Usage

```
computeSingVarInt(
  object,
  which.z,
  qs.diff = c(0.25, 0.75),
  qs.fixed = c(0.25, 0.75),
  nsamples = 1000,
  ...
)
```

Arguments

object	an object of class gpr obtained from gvagpr function
which.z	vector indicating which variables (columns of Z) for which the summary should be computed
qs.diff	vector indicating the two quantiles at which to compute the single-predictor risk summary
qs.fixed	vector indicating the two quantiles at which to fix all of the remaining exposures in Z
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
...	unused argument

Value

a data frame containing the (posterior mean) estimate and posterior standard deviation of the single-predictor risk measures

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
```

```
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
risks.int <- computeSingVarInt(fout, which.z = 1:3, nsamples = 300)
risks.int$summary
plot(risks.int)
```

computeSingVarRisk	<i>Single Variable Risk Summaries</i>
--------------------	---------------------------------------

Description

Compute summaries of the risks associated with a change in a single variable in Z from a single level (quantile) to a second level (quantile), for the other variables in Z fixed to a specific level (quantile)

Usage

```
computeSingVarRisk(
  object,
  which.z,
  qs.diff = c(0.25, 0.75),
  q.fixed = c(0.25, 0.5, 0.75),
  nsamples = 1000,
  ...
)
```

Arguments

object	an object of class gpr obtained from gvagpr function.
which.z	vector indicating which variables (columns of Z) for which the summary should be computed
qs.diff	vector indicating the two quantiles q_{-1} and q_{-2} at which to compute $f(z_{-}\{q_2\}) - f(z_{-}\{q_1\})$
q.fixed	vector of quantiles at which to fix the remaining predictors in Z
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
...	unused argument

Value

a data frame containing the (posterior mean) estimate and posterior standard deviation of the single-predictor risk measures

References

Bobb J (2023). `_bkmr: Bayesian Kernel Machine Regression_`. R package version 0.2.2, <<https://github.com/jenfb/bkmr>>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
risks.singvar <- computeSingVarRisk(fout, which.z = 1:3, nsamples = 300)
risks.singvar$summary
plot(risks.singvar)
```

extractELBO

Extract ELBO from gpr, ggpr, or gpmim model fits

Description

Compute the evidence lower bound (ELBO) at the fitted variational approximation for class "gpr" or "gpmim".

Usage

```
extractELBO(object, nsamples = 1000)
```

Arguments

object	an object of class gpr or gpmim.
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000.

Details

The value is a Monte-Carlo estimate of the bound at the fitted q , averaged over `nsamples` fresh draws seeded from the fit, so the same object always gives the same number. It is not the bound the optimiser recorded along the way: that trace is `object$elbo_trace`, and its moving average, which the stopping rule reads, is `object$elbo_smooth`.

Which quantity the trace is differs by model. A Gaussian fit (`gvagpr`, `gvagpim`, with or without random effects or mini-batches) records the expected log joint without the entropy of the variational distribution, so at the same parameters its trace sits above this value by minus the entropy; a Poly-Gamma fit records the full bound. The trace is also a moving average taken while the optimiser was still climbing, so on a short run it lags well below the bound at the returned parameters. This function returns the full bound for every model.

The Monte-Carlo error depends on the model. On a Gaussian index fit the single-draw standard deviation of the bound was measured at about 800 nats, which makes the default `nsamples` good to about 25 nats; compare fits at the same `nsamples` and raise it when the difference is small. The Poly-Gamma bounds are far less noisy.

For a multiple-index fit with a Poly-Gamma outcome (`gvaggpim`) the value is the collapsed bound the fit climbed – the inducing values integrated out, the random effects, if any, as coordinates – evaluated the same way. Each draw solves the bound's inner fixed point for the Poly-Gamma parameter, so this is slower per draw than the other bounds.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr, gvagpmim, gvaggpmim

extractPostSamps

*Extract Posterior Samples from gpr, ggpr, or gpmim model fits***Description**

Generate the posterior samples for class "gpr" or "gpmim"

Usage

extractPostSamps(object, nsamples = 1000)

Arguments

object	an object of class gpr or gpmim.
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000.

Value

a data frame including posterior samples for all parameters. If family is gaussian, the posterior samples contain β , σ^2 , λ_f , and γ (for an index model, ρ , ψ , γ and α in place of γ). For a gvaggpr fit the posterior samples also contain the inducing values $\bar{f}_j, j = 1, \dots, \bar{n}$; a gvaggpmim fit has no such block, its bound integrating the inducing values out, and returns none. If object\$id is not NULL, the data frame also includes $b_i, i = 1, \dots, N$.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr, gvagpmim, gvaggpmim

extractRandomEffects	<i>Extract the Random Effects of a Fitted Model</i>
----------------------	---

Description

Returns the variational posterior means and standard deviations of the random intercepts (and slopes, when `random.slope` was given) of a model fitted by `gvagpr`, `gvaggpr`, `gvagpmim` or `gvaggpmmim` with a cluster indicator `id`, one row per cluster, labelled by the levels of `id`.

Usage

```
extractRandomEffects(object)
```

Arguments

`object` an object of class `gpr` or `gpmim` fitted with `id`.

Value

a list with `mean` and `sd`, each an `nID` by `q` matrix whose rows are the clusters and whose columns are the intercept and, when present, the slopes; and `Sigma.b`, the estimated covariance matrix of the random effects, evaluated at the variational mean.

Author(s)

Seongil Jo

See Also

`gvagpr`, `gvaggpr`, `gvagpmim`, `gvaggpmmim`

<code>fitted.gpmim</code>	<i>Extract GPMIM Fitted Values</i>
---------------------------	------------------------------------

Description

fitted is a generic function which extracts fitted values of nonparametric part from an object of class `"gpmim"`

Usage

```
## S3 method for class 'gpmim'
fitted(
  object,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

object	an object of class <code>gpmim</code> .
nsamples	(positive integer) number of draws from the variational posterior. Used only when <code>variance = "total"</code> ; the fitted surface itself needs no draws. Default value is 1000.
cov	logical. If TRUE (the default) the covariance matrix of the nonparametric part is returned as well. This matrix is n by n , so for large samples it may dominate the memory used by the call; set <code>cov = FALSE</code> to return the fitted means only.
variance	one of "conditional" (the default) or "total", choosing which covariance <code>fcov</code> reports. See the Value section.
...	unused argument.

Value

fmean	the exposure–response surface at the posterior median of the parameters, $E[f y, \hat{\theta}]$, where $\hat{\theta}$ is the componentwise median of the variational posterior. This is a plug-in and not the posterior mean $E[f y]$, which would average $E[f y, \theta]$ over $q(\theta)$. Returned in the order of the data supplied to the fitting function (also for random-effects fits, whose rows are stored sorted by cluster).
fcov	NULL if <code>cov = FALSE</code> . With <code>variance = "conditional"</code> , the covariance of f at that same $\hat{\theta}$. It conditions on the parameters, so it drops $\text{Var}_{\theta}(E[f y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$. It is usually the narrower of the two, but not necessarily at every point: the difference between them is not positive semi-definite. With <code>variance = "total"</code> , the posterior covariance of f itself, obtained from the law of total variance over nsamples draws; that term has no closed form, so this option costs one $n \times n$ factorisation per draw and is much slower.
an object of class "gprfit", which has the associated method:	
* <code>plot</code> (i.e., <code>plot.gprfit</code>)	

Author(s)

Seongil Jo

See Also`gvagpmim`, `gvaggpmim`

fitted.gpr

*Extract GPR and GGPR Model Fitted Values***Description**

fitted is a generic function which extracts fitted values of nonparametric part from an object of class "gpr"

Usage

```
## S3 method for class 'gpr'
fitted(
  object,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

<code>object</code>	an object of class <code>gpr</code> .
<code>nsamples</code>	(positive integer) number of draws from the variational posterior. Used only when <code>variance = "total"</code> ; the fitted surface itself needs no draws. Default value is 1000.
<code>cov</code>	logical. If <code>TRUE</code> (the default) the covariance matrix of the nonparametric part is returned as well. This matrix is n by n , so for large samples it may dominate the memory used by the call; set <code>cov = FALSE</code> to return the fitted means only.
<code>variance</code>	one of <code>"conditional"</code> (the default) or <code>"total"</code> , choosing which covariance <code>fcov</code> reports. See the Value section.
<code>...</code>	unused argument.

Value

<code>fmean</code>	the exposure–response surface at the posterior median of the parameters, $E[f \mid y, \hat{\theta}]$, where $\hat{\theta}$ is the componentwise median of the variational posterior. This is a plug-in and not the posterior mean $E[f \mid y]$, which would average $E[f \mid y, \theta]$ over $q(\theta)$. Returned in the order of the data supplied to the fitting function (also for random-effects fits, whose rows are stored sorted by cluster).
<code>fcov</code>	NULL if <code>cov = FALSE</code> . With <code>variance = "conditional"</code> , the covariance of f at that same $\hat{\theta}$. It conditions on the parameters, so it drops $\text{Var}_{\theta}(E[f \mid y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$. It is usually the narrower of the two, but not necessarily at every point: the difference between them is not positive semi-definite. With <code>variance = "total"</code> , the posterior covariance of f itself, obtained from the law of total variance over $nsamples$ draws; that term has no closed form, so this option costs one $n \times n$ factorisation per draw and is much slower.

an object of class `"gprfit"`, which has the associated method:

* `plot` (i.e., `plot.gprfit`)

Author(s)

Seongil Jo

See Also

`gvagpr`, `gvaggpr`

gvaggpmm

*Gaussian Variational Approximation to Generalized Gaussian Process Multiple Index Model***Description**

Fits the Bayesian kernel machine multiple index model for binomial and negative binomial outcomes by Gaussian variational approximation with Polya-Gamma augmentation, with optional cluster random intercepts and slopes.

Usage

```
gvaggpmm(
  y,
  X,
  Z,
  m.index = NULL,
  id = NULL,
  random.slope = NULL,
  family = binomial(),
  priors = list(),
  inits = list(),
  covstr = c("fullrank", "diagonal", "blockdiag", "sparseprec"),
  control = list(),
  minibatch = TRUE,
  verbose = TRUE,
  seed = sample.int(.Machine$integer.max, 1)
)
```

Arguments

<code>y</code>	a vector of outcomes: 0/1 for binomial, non-negative integer counts for negbinomial.
<code>X</code>	a matrix of covariates that enter the model linearly.
<code>Z</code>	exposures: either a matrix, with <code>m.index</code> giving the grouping, or a list with one element per index.
<code>m.index</code>	a list of integer positions or exposure names, one element per index, used when <code>Z</code> is a matrix. Ignored, with a warning, when <code>Z</code> is a list.
<code>id</code>	optional vector of length <code>n</code> of cluster indicators for a model with random effects (a random intercept and, with <code>random.slope</code> , random slopes), as in <code>gvaggr</code> and <code>gvagpm</code> . <code>NULL</code> fits the fixed-effects model. The rows of a random-effects fit are stored sorted by cluster; <code>fitted</code> returns values in the order the data were given.
<code>random.slope</code>	index of the column(s) of <code>X</code> that carry a random slope. Read only together with <code>id</code> ; without it a warning is given and the fixed-effects model is fitted.
<code>family</code>	<code>binomial()</code> or <code>negbinomial()</code> ; the dispersion of the latter is estimated rather than fixed. A count outcome puts the inner fixed point for the Polya-Gamma parameter c under more strain than a binary one, because the linear predictor it works on is $\eta - \log r$; see <code>control\$ci</code> and the section below.

priors	a list; lengthscale is 'horseshoe' (the default) or 'indep.normal', and the remaining elements give the hyperparameters, with their defaults in parentheses. $\text{alam}(0.1)$ and $\text{blam}(0.01)$ are the shape and rate of the gamma prior on the kernel amplitude λ_f . Under 'horseshoe', $\text{lam0}(1)$ is the scale of the half-Cauchy prior on each local scale λ_{ml} , $\text{tau0}(1)$ that of the per-index scale τ_m and $\text{taur0}(1)$ that of the global scale; under 'indep.normal', $\text{tau0}(10)$ is the standard deviation of the normal prior on each ρ_{ml} . For a count outcome $\text{s_theta}(1)$ is the scale of the half-normal prior on $1/\sqrt{r}$, as in <code>gvaggpr</code> . With random effects $\text{vtaub}(100)$ is the variance of the normal prior on the free elements of the Cholesky factor of Σ_b , as in <code>gvaggpr</code> . 'horseshoe' is the same prior <code>gvagpmim</code> uses, local scales and all: integrating the inducing values out frees the parameter budget to carry them.
inits	a list of starting values.
covstr	structure of the variational covariance. Without random effects: "fullrank" (the default) or "diagonal". Both recover the index equally well, but a diagonal covariance understates the posterior spread of the index weights by about a factor of two, and those weights are usually what is read; "diagonal" is the cheaper choice when only the fitted surface is wanted; "blockdiag" and "sparseprec" are treated as "fullrank". With random effects: "diagonal", "blockdiag" (one block per cluster and one for the shared parameters) or "sparseprec" (a sparse precision matrix that adds the dependence between each cluster and the shared parameters, updated by the natural gradient of Tan (2025)), as in <code>gvaggpr</code> ; an unrestricted factor is not offered there, so "fullrank" becomes "blockdiag", silently when <code>covstr</code> was left at its default and with a warning when it was asked for.
control	a list of options for the optimiser, as in <code>gvaggpr</code> (<code>max_iter</code> , <code>alpha0_mu</code> , <code>alpha0_L</code> , <code>beta</code> , <code>nws</code> , <code>nsp</code>), including <code>ninpts</code> and <code>knots</code> for the inducing points and <code>batch.size</code> for the mini-batches. For a count outcome <code>batch.size</code> defaults to 100 and <code>nbatch</code> may be given instead, the two having to agree. For a binary outcome the split is not a setting: it is fixed at 150, and a <code>batch.size</code> or <code>nbatch</code> that is supplied is refused with a warning. <code>cit(30)</code> caps the inner fixed-point iteration for the Polya-Gamma parameter c inside the bound; the iteration exits as soon as it converges, which small binary samples do in about ten passes, while counts and larger binary samples run to the cap on most updates. The fraction that did is reported as <code>c.unconverged</code> ; wherever it was measured, raising the cap did not move the answer. See <i>What has been checked</i> below.
minibatch	TRUE or FALSE (default TRUE): the likelihood is evaluated on a batch and the priors are scaled by the batch fraction, as in <code>gvagpmim</code> . The inducing-point kernel does not depend on the batch, so only the data sums are estimated from it; the cost per update stops depending on n . How large a batch has to be before this is free depends on the family: a binary outcome is held at 150 and refuses a smaller split, counts default to 100 and may be changed, and a binary outcome below five hundred observations is warned about rather than batched silently. See <i>Mini-batches and the sample size</i> below. The option does not apply to the random-effects models, which use the whole sample at every update.
verbose	print progress.
seed	random seed.

Details

The model is the multiple-index kernel machine of `gvagpmim` with a Polya-Gamma likelihood in place of the Gaussian one. Exposures enter through M indices, the m th being a weighted combi-

nation of the exposures in group m , and the kernel is $K_{ij} = \exp(-\sum_m \{(z_{im} - z_{jm})'\psi_m\}^2)$ with $\psi_m = D_m^{-1}\rho_m$. The last coordinate of each ρ_m is positive, which is what encodes $\sum_l \alpha_{ml} \geq 0$ and identifies the index.

Two length-scale priors are offered, the same two `gvagpmm` offers. 'horseshoe' gives each ρ_{ml} a normal with a local scale $\lambda_{ml} \sim C^+(0, s)$, s being priors\$lam0, on top of a per-index τ_m and a global τ_γ , so shrinkage acts on an exposure, on an index and overall; the last coordinate of each index is restricted to the positive half, which is what identifies the index. 'indep.normal' adds no coordinates beyond ρ . Carrying the local scales is affordable here because the inducing values are integrated out of the bound rather than estimated – under the older formulation, which kept them, the local scales had to be integrated out instead to keep the optimisation stable.

Value

an object of class `gpmim` carrying family. A random-effects fit also carries `id`, `id.labels`, `nID`, `q`, `U`, `order` (the permutation that sorted the rows by cluster) and, with `random.slope`, `random.slope`; the cluster effects and their covariance are read with `extractRandomEffects`. Beyond the fields a `gvagpmm` fit has, it reports two diagnostics, neither of which raises a warning: `c.unconverged`, the fraction of updates on which the inner fixed point for c was not reached within `control$cit`, and `cond.K_IP`, the condition number of the inducing-point kernel at the fitted index weights. A large condition number is the normal state of an RBF kernel over a few dozen inducing points in a low-dimensional score space, not a symptom; the two ways the kernel actually loses information are caught by the kernel check at the end of the fit.

Mini-batches and the sample size

A Bernoulli observation carries much less information than a count, and the index model feels the difference. On a moderate-signal design of six exposures in three groups, comparing each batched fit with the full-data fit on the same data, a binary outcome at four hundred observations gives a shrunken surface at every batch size the code admits – the coarsest split available there is two batches, and it is still clearly worse than the full data – while from five hundred observations a batch of 150 matches the full data on the index weights and on the selection probabilities, at a quarter of the cost. A smaller split stays behind the full data well past a thousand observations: at $n = 500$ a batch of 100 gives roughly twice the error of a batch of 150 and misclassifies exposures the larger batch gets right.

Two things follow. The split for a binary outcome is **fixed** at 150 rather than defaulted – a `batch.size` or `nbatch` passed in `control` is refused with a warning, because there is nothing to gain by going below it. And asking for mini-batches with fewer than five hundred binary observations warns and suggests `minibatch = FALSE`; that one does not override the request, since no batch size would rescue it and the cost of the full data is the user's to weigh.

A count outcome shows none of this. At the same four hundred observations a batch of fifty already reproduces the full-data surface (regression of the true surface on the fitted one: slope 0.999 against 0.998 for the full data), so the default there stays at 100 and no sample-size warning is given.

Both statements are about the index model, and its default is the only one that changed. The basic model `gvagpmm` was measured the same way and does not shrink: on the same design at two hundred and at four hundred observations, batches of 100 and of 50 reproduce the full-data surface – slope 0.916 and amplitude ratio 0.919 at a batch of 50 against 0.969 and 0.892 for the full data – where the index model at the same batch of 50 returns a slope of 6.2 and an amplitude eleven per cent of the truth. Its batch size therefore stays at 100 and it carries no sample-size warning.

The difference is structural rather than numerical, and worth stating because it says where else to look. The index model integrates the inducing values out of its bound, so the amplitude of the surface is rebuilt afterwards from the fitted hyperparameters and inherits any bias a batch puts

in them; the basic model carries those inducing values as a variational block, which the batched gradient estimates perfectly well. Neither model has a gradient error – both were checked against numerical differentiation, and the index model shrank anyway, which is why this went unnoticed for so long.

What has been checked, and on what

A binary outcome is the better-tested path. Over sixty simulated data sets it recovers the index weights to a mean absolute error of about 0.045, and the fit has been compared against a Stan posterior for the same model and against a second machine with a different compiler and BLAS.

That record carries two qualifications, both learned late. Those data sets carry a strong signal; and the agreement that used to be reported here between the eight combinations of prior, variational covariance and mini-batching was measured at four hundred observations with a batch of one hundred – a configuration this function now refuses for a binary outcome, and the worst one in the table under *Mini-batches and the sample size*. What was compared across those combinations was the index weights and the inclusion probabilities, and both are scale-free: they do not change when the fitted surface is uniformly too small, so they could not have seen the shrinkage the mini-batched fits turn out to have at a moderate signal. Read that agreement as reproducibility across combinations, which it is, and not as evidence that mini-batching was free, which it was not.

A count outcome is newer, and rests on a hundred simulated data sets of four hundred observations across three designs, chosen to move the level and the overdispersion together rather than one design repeated:

design	reps	mean count	slope	R-sq	dispersion	true
A	60	8.9	1.011	0.974	2.11	2
B	20	17.7	1.010	0.948	4.93	5
C	20	3.6	0.993	0.951	1.03	1

The surface comes back calibrated in amplitude as well as in shape – the ratio of fitted to true standard deviation is 0.96 to 0.99, and the slope of the true surface on the fitted one has a 95 per cent spread of 0.92 to 1.12 on design A. The linear coefficient averages 0.289 to 0.307 against a true 0.300 and is covered by its interval in 95 to 100 per cent of fits. The dispersion is recovered over a fivefold range. Under 'horseshoe' the inclusion probabilities misclassify one exposure in six hundred decisions. The designs are not tame: the heaviest draw on design A reaches a count of 955. The two length-scale priors are indistinguishable on the surface, so the shrinkage one costs nothing for the selection it adds.

What has NOT been established for counts: behaviour at means in the hundreds, and agreement with an exact posterior of the kind the binary path was compared against.

Random effects with a binary outcome were checked on a hundred clusters of twenty with a random intercept of variance 0.36, ten replicates: index weights to a mean absolute error of 0.02 and, under 'horseshoe', no misclassified inclusion probability; the true surface regressing on the fitted one with slope 1.09 under "blockdiag" and "sparseprec" (1.19 under "diagonal"); coefficient intervals covering in 80 to 90 per cent of fits; the random-intercept variance at 0.31 to 0.37; and the cluster effects correlated 0.72 with the truth – the numbers gvaggpr gives when fitted on the true index scores. A random slope on a time-varying covariate inherits gvaggpr's over-estimation of the slope variance under weak within-cluster information (0.80 to 1.06 against a true 0.25, where gvaggpr gives 0.81). Against Stan posteriors for the same model on the same inducing points: with forty clusters of ten and a moderate surface, coefficients, index weights and cluster effects agree (means within 0.03, standard deviations in a ratio of 0.9 to 1.1, cluster-effect means correlated 0.999) and the random-intercept variance sits within one posterior standard deviation of the exact

one; with twenty clusters of ten and a strong surface the means still agree but the cluster-effect standard deviations are about twice Stan's and the random-intercept variance is 0.25 to 0.32 where the exact posterior sits near zero – with so little information the exact posterior follows the prior's pull towards zero and a Gaussian factor cannot. The count outcome with random effects was checked the same way (negative binomial with dispersion 2 and a mean count near eleven, the same clusters, ten replicates, 130 fits): index weights to a mean absolute error of 0.02 with no misclassified exposure, the surface regressing with slope 1.02 to 1.03, the random-intercept variance at 0.33 to 0.35 and the dispersion at 1.96 to 2.00 (truth 0.36 and 2), each within 0.01 of gvaggpr on the true scores; the random-slope variance is not over-estimated there (0.28 to 0.32 against 0.25). Against a Stan posterior on the same inducing points (forty clusters of ten) the coefficients, weights, dispersion, random-intercept variance and cluster effects agree in mean to 0.02 with standard-deviation ratios of 0.9 to 1.2 under "blockdiag" and "sparseprec". Fitting such counts without `id` leaves the surface and the weights intact and pushes the dispersion down (1.1 to 1.3 for a true 2): the cluster effects are absorbed as over-dispersion. In three of thirty random-slope fits the index-level inclusion probability of an active index fell to 0.2 to 0.3 while its exposures stayed above 0.87; read the exposure-level probabilities alongside the index-level ones.

`extractELB0` evaluates this bound at the fitted parameters, with or without random effects. An R-side replication of it agrees to $2e-13$, and it sits 0.2 to 1.2 per cent above the smoothed in-loop trace on the package's full-data test fits, the trace being a moving average taken while the optimiser was still climbing. On a mini-batched fit the trace is the scaled subsample estimate and the two are not the same number.

A count outcome also strains the bound's inner fixed point for the Polya-Gamma parameter c , because it works on $\eta - \log r$ rather than η . `out$c.unconverged` reports the fraction of updates that did not reach it within `control$cit`; it runs high on count fits and, on everything measured, does not move the answer. The cap defaults to 30, where the answer is already settled: raising it to 1000 changed the index weights by $2e-4$ and the dispersion by 0.0002, and cost 36 times the time.

Random effects

The random effects enter the linear predictor, $\eta = X\beta + Ub + f(z)$, with $b_i \sim N(0, \Sigma_b)$ per cluster and the Cholesky factor of Σ_b given a normal prior on its free elements, exactly as in `gvaggpr` and `gvagpmim`. They are variational coordinates, and the inducing values are still integrated out of the bound: b enters the bound only through the linear predictor, so the collapsed bound of the fixed-effects model carries over with $X\beta$ replaced by $X\beta + Ub$, plus the prior on b and its covariance. The latent surface is rebuilt from the fitted parameters with the cluster effects in the linear predictor, so `fitted` returns the population-level surface and not the surface plus the cluster effects. These fits use the whole sample at every update; mini-batches do not apply.

How the bound is built

The latent values at the inducing points are integrated out of the bound rather than carried as variational coordinates. That matters here in a way it does not for `gvaggpr`: an index model combines the exposures *before* the kernel, so ρ moves the inducing points themselves and a value attached to one stops meaning the same thing. Kept as coordinates they and ρ sit on a curved ridge that a joint Gaussian cannot follow, and the fit settles on a degenerate solution in which the free coordinates of each index are driven to zero, only the exponentiated one survives, and the kernel collapses. Integrating them out removes the ridge; the latent process is then recovered in closed form when a surface is asked for, which is one approximation fewer than reading a variational $q(f_{IP})$ would be.

The starting direction for ρ is taken from a quadratic logistic fit (an index enters its quadratic part as $\rho'zz'\rho$, so the leading eigenvector of the within-index block points along ρ). It costs a few thousandths of a second and only has to be roughly right: the scale does not matter and a poor direction is recovered from. Nothing here is user-facing.

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpr, computePPIPs

Examples

```
## Six exposures in three indices; only the first two carry signal, and
## within the first only As and Cd do. This is the design the package's own
## tests use.
set.seed(1)
n <- 150
g1 <- matrix(runif(n * 3, -1, 1), n, 3)
g2 <- matrix(runif(n * 2, -1, 1), n, 2)
g3 <- matrix(runif(n * 1, -1, 1), n, 1)
Z <- cbind(As = g1[, 1], Cd = g1[, 2], Pb = g1[, 3],
           p1 = g2[, 1], p2 = g2[, 2], s1 = g3[, 1])
X <- cbind(x1 = rnorm(n))
m.index <- list(1:3, 4:5, 6)

h <- (2 * as.numeric(g1 %*% c(0.8, 0.6, 0)) + 0.5)^2 -
      (2 * as.numeric(g2 %*% c(0.7, 0.7)))^2
eta <- 0.3 * X[, 1] + h - mean(h)
y <- rbinom(n, 1, plogis(eta))

## Below five hundred observations a binary index model is fitted on the
## whole sample. The iteration caps in `ctl` keep this example short;
## drop them for a real analysis and let the stopping rule decide.
ctl <- list(ninpts = 20, max_iter = 2000, nws = 500, nsp = 30)

fout <- gvaggpmmim(y, X, Z, m.index, family = binomial(), control = ctl,
                  minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fout)      # 'alpha.' rows are the index weights, 'gamma.' the scales
plot(fout)         # the smoothed bound; read the plateau, not the climb
computePPIPs(fout, nsamples = 300) # needs a shrinkage lengthscale (the default)

## the latent surface at the observed exposures, and at new ones
fitted(fout, cov = FALSE)$fmean[1:5]
predict(fout, Z_new = Z[1:3, ], cov = FALSE)$fmean

## a count outcome
yc <- rnbinom(n, size = 2, mu = exp(1 + 0.3 * eta))
fc <- gvaggpmmim(yc, X, Z, m.index, family = negbinomial(), control = ctl,
                  minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fc)        # 'theta' is the estimated dispersion

## a random intercept per cluster, under the sparse precision structure
id <- rep(1:15, each = 10)
yr <- rbinom(n, 1, plogis(eta + rnorm(15, sd = 0.6)[id]))
fr <- gvaggpmmim(yr, X, Z, m.index, id = id, family = binomial(),
                  covstr = "sparseprec", control = ctl, verbose = FALSE, seed = 1)
extractRandomEffects(fr)$Sigma.b

## Mini-batches are the default from five hundred observations; a binary
```

```
## index model splits at 150 and takes no batch.size (see Details).
```

gvaggpr

Gaussian Variational Approximation to Generalized Gaussian Process Regression

Description

Fits the Bayesian kernel machine regression for generalized linear model using Gaussian variational approximation algorithm.

Usage

```
gvaggpr(
  y,
  X,
  Z,
  id = NULL,
  random.slope = NULL,
  family = binomial(),
  priors = list(),
  inits = list(),
  covstr = c("diagonal", "fullrank", "blockdiag", "sparseprec"),
  control = list(),
  minibatch = TRUE,
  verbose = TRUE,
  seed = sample.int(.Machine$integer.max, 1)
)
```

Arguments

y	a vector of response of length n.
X	an n-by-p matrix of covariates for parametric term. Should not contain an intercept.
Z	an n-by-M matrix of predictor variables to be included in nonparametric part.
id	optional vector (of length n) of grouping factors for fitting a model with random effects (including both a random intercept and a random slope). If NULL then no random effects will be included.
random.slope	a column index of the matrix (X) including covariates for random slope. If NULL and id is given, the model considers the random intercept only. Two caveats, both measured on simulated binary outcomes with 100 clusters. (i) Fitting a slope the design does not need inflates its variance component: against a true $\text{var}(b_1)$ of 0.64 the fit returns 1.03 at $n_i = 5$ and 0.70 at $n_i = 50$ (glmer with an offset gives 0.62 and 0.55). The excess shrinks as information accumulates and is a property of the Gaussian variational family under weak identification, not of the Polya-Gamma step. Point estimates, selection, the surface and $\text{var}(b_0)$ are unaffected, so the cost of an unneeded slope falls on that one number. Whether the slope is needed is predicted by $v = \text{var}(b_1) \overline{(z - \bar{z}_i)^2}$: an intercept-only fit attenuates by about $(1 + 0.346v)^{-1/2}$. (ii) Fixed-effect posterior intervals from

	a slope model are slightly narrow. The coupling row of the sparse precision converges last, so the reported sd of a fixed effect falls short of its closed form under the bound; point estimates and the variance components are unaffected.
family	a description of the error distribution and link function. Either <code>binomial()</code> with the logit link, or <code>negbinomial()</code> with the log link, in which case the linear predictor is the log mean, as in <code>MASS::glm.nb</code> . The dispersion is always estimated; a starting value may be given through <code>inits\$theta</code> . <code>MASS::negative.binomial(theta =)</code> is also accepted, its <code>theta</code> taken as that starting value, with a warning saying so.
priors	a list giving the prior information. The list includes the following parameters (with default values in parentheses): <code>alam(10)</code> and <code>blam(1)</code> giving the shape and rate of the gamma prior on λ_f , <code>tau0</code> giving the scale of the prior on the inverse length-scales (1 under 'betaprime', 10 under 'normal' and 'indep.normal'), <code>s_theta(1)</code> giving the scale of the half-normal prior placed on $1/\sqrt{\tau}$, the standard deviation of the multiplicative random effect in the Poisson-gamma representation of the negative binomial. The element <code>lengthscale</code> selects the prior on the inverse length-scale parameters and is one of 'normal' (a single length-scale shared by all exposures), 'indep.normal' (one length-scale per exposure, the default when <code>ncol(Z) > 1</code>) or 'betaprime' (one per exposure with a continuous shrinkage prior, which performs variable selection). 'shrinkage' is accepted as a deprecated alias for 'betaprime'.
inits	a named list of starting values for the variational means, given by name rather than by position, so that the order in which the blocks are stored does not matter. Every element is given on the natural scale and is logged internally where the parameter is positive; a scalar is recycled over a block. The names are <code>beta</code> (the regression coefficients), <code>lengthscale</code> (the inverse length-scales γ), <code>lambda</code> (the scale of the Gaussian process), <code>taur</code> (the global shrinkage scale, for <code>lengthscale = "betaprime"</code>), and, for the negative binomial family, <code>theta</code> (the dispersion). Anything not named starts at zero on the working scale. The inducing values and the random-effect covariance are set internally and are not exposed.
covstr	Either "diagonal" (the default), "fullrank", "blockdiag", or "sparseprec", indicating which covariance structure of variational distribution is used. The "diagonal" option uses a fully factorized Gaussian for the approximation, and is available for every model. For a model without random effects, the "fullrank" option uses a Gaussian with an unrestricted lower triangular Cholesky factor. For the mixed effect models the parameter vector is partitioned by cluster and an unrestricted factor is not offered: the "blockdiag" option uses a Gaussian whose covariance matrix is block diagonal, with one block per cluster and one block for the parameters shared by all clusters, and the "sparseprec" option uses a Gaussian with a sparse precision matrix, which adds the dependence between each cluster and the shared parameters. An option that does not apply to the model at hand falls back to the closest one that does, with a warning: "fullrank" becomes "blockdiag" when random effects are present, and "blockdiag" and "sparseprec" become "fullrank" when they are not.
control	a named list of parameters to control the algorithm's behavior. The list includes the following parameters (with default values in parentheses): <code>max_iter(100000)</code> giving the maximum number of iterations, <code>batch.size(100)</code> giving the size of each mini-batch when <code>minibatch = TRUE</code> , or equivalently <code>nbatch</code> giving their number (supplying both is an error unless they agree), <code>alpha0_mu(1e-3)</code> and <code>alpha0_L(1e-4)</code> giving the base step sizes for the mean and Cholesky

parameters, respectively. For the random-effects models, which use the whole sample at each iteration, `alpha0_L` defaults instead to $3e-3$ with `covstr = "sparseprec"` and to $1e-3$ otherwise; a value given in `control` is used as given, `beta` (0.9) giving the exponential moving-average coefficient used to smooth the (normalized) natural-gradient direction, `nws` (2500) giving rolling window size for calculating the moving average of the lower bounds, `nsp` (100) giving the maximum patience parameter, The loop stops once the rolling mean of the lower bound has failed to improve on its best value for `nsp` consecutive updates, and the message printed on stopping – "The algorithm has converged" – reports that this counter was reached, not that the bound has settled. A small `nsp` can therefore stop on a run of unlucky updates and still report success. Read the plateau in `plot()` before trusting a fit that stopped early, and raise `nsp` if the bound is still climbing. `ninpts` (50) giving the number of inducing points for sparse Gaussian process prior. `knots` (`NULL`) giving the inducing points for sparse Gaussian process prior.

<code>minibatch</code>	TRUE or FALSE (default TRUE): if TRUE the data are split into batches of <code>control\$batch.size</code> and <code>max_iter</code> counts epochs rather than updates. A split that would give fewer than two batches is not a mini-batch, so the full data are used instead and a warning is given; with the default batch size of 100 that is every sample smaller than 200. The option does not apply to the random effects models.
<code>verbose</code>	TRUE or FALSE: flag indicating whether to print intermediate diagnostic information during the model fitting.
<code>seed</code>	The seed for random number generation. The default is generated from 1 to the maximum integer supported by R on the machine.

Details

Jo et al. (2026+) proposed the Bayesian semiparametric generalized linear model with sparse Gaussian process prior based on the Radial basis function (RBF) kernel:

$$\begin{aligned}
 y_i &\sim \text{Bern}(p_i), \log(p_i/(1+p_i)) = x_i^\top \beta + f(z_i), \\
 f &= (f(z_1), \dots, f(z_D))^\top \sim GP(K^{-1} \bar{f}, \sigma^2 \lambda_f \tilde{K}), \quad z_i = (z_{i1}, \dots, z_{iM})^\top, \\
 \bar{f} &= (f(\bar{z}_1), \dots, f(\bar{z}_{\bar{D}}))^\top \sim GP(0, K_{\bar{D}}),
 \end{aligned}$$

where $\bar{D} \ll D$, $K = K_D K_{\bar{D}\bar{D}}^{-1}$, $\tilde{K} = K_D - K_{D\bar{D}} K_{\bar{D}\bar{D}}^{-1} K_{\bar{D}D}$, and the kernel denotes the RBF kernel given as

1) Equal lengthscale parameter:

$$K_D = \left(\exp \left(-\gamma \sum_{m=1}^M \|z_i - z_j\|^2 \right) \right)_{i,j=1}^D$$

2) Varying lengthscale parameters:

$$K_D = \left(\exp \left(- \sum_{m=1}^M \gamma_m \|z_i - z_j\|^2 \right) \right)_{i,j=1}^D$$

For the parameters, the following priors are used:

$$\pi(\beta) \propto 1,$$

$$\pi(\lambda_f) = \text{Gamma}(a_\lambda, b_\lambda),$$

1) Normal prior:

$$\pi(\gamma) = N_+(0, \tau_0^2)$$

2) Independent Normal priors:

$$\pi(\gamma_m) = N_+(0, \tau_0^2), \quad m = 1, \dots, M$$

3) Beta-prime (continuous shrinkage) prior:

$$\gamma_m \mid \tau_\gamma \sim \text{BetaPrime}\left(\frac{1}{2}, \frac{1}{2}; \text{scale} = \tau_\gamma^2\right), \quad m = 1, \dots, M,$$

$$\pi(\gamma_m \mid \tau_\gamma) \propto \gamma_m^{-1/2} (1 + \gamma_m/\tau_\gamma^2)^{-1}, \quad \gamma_m > 0,$$

$$\tau_\gamma \sim \mathcal{C}^+(0, \tau_0),$$

where τ_0 is a positive constant specified by the user.

For more details, see Jo and Lee (2026+).

Value

an object of class "gpr", which has the associated methods:

- * extractELBO
- * fitted (i.e., fitted.gpr)
- * summary (i.e., summary.gpr)
- * predict (i.e., predict.gpr)
- * plot (i.e., plot.gpr)
- * extractRandomEffects, for a fit with random effects

For a fit with random effects the rows of y, X, Z and U in the object are sorted by cluster, as the optimiser requires; order is the permutation that was applied and id.labels the cluster labels in the order of the integer codes in id. fitted returns its values in the order the data were supplied.

Author(s)

Seongil Jo and Woojoo Lee

References

- Jo, S., and Lee, W. (2025+), "Gaussian variational approximation to Bayesian kernel machine regression for Logistic models", *preprint*.
- Jo, S., Ye, S., Hanh, G., and Lee, W. (2026), "Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior", *Statistics and Computing*.
- Polson, N. G., Scott, J. G., and Windle, J. (2013), "Bayesian Inference for Logistic Models using Polya-Gamma Latent Variables", *Journal of the American Statistical Association*, 108, 1339-1349.
- Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.
- Tan, L. S. L. and Nott D. J. (2018), "Gaussian variational approximation with sparse precision matrices", *Statistics and Computing*, 28, 259-275.
- Tan, L. S. L. (2025), "Analytic natural gradient updates for Cholesky factor in Gaussian variational approximation", *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 00, 1-27.

See Also

extractELBO, fitted.gpr, predict.gpr, plot.gpr, summary.gpr

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
## a binary outcome
yb <- rbinom(n, 1, plogis(0.5 * X[, 1] + h))
fb <- gvaggpr(yb, X, Z, family = binomial(), control = list(ninpts = 30),
              minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fb)
plot(fb)
extractELBO(fb, nsamples = 200)

## a count outcome; 'theta' in the summary is the estimated dispersion
yc <- rbinom(n, size = 2, mu = exp(1 + 0.3 * h))
fc <- gvaggpr(yc, X, Z, family = negbinomial(), control = list(ninpts = 30),
              minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fc)

## a random intercept and a random slope on the covariate
id <- rep(1:12, each = 10)
b0 <- rnorm(12, sd = 0.6)
yr <- rbinom(n, 1, plogis(0.5 * X[, 1] + h + b0[id]))
fr <- gvaggpr(yr, X, Z, id = id, random.slope = 1, family = binomial(),
              covstr = "sparseprec", control = list(ninpts = 30),
              verbose = FALSE, seed = 1)
extractRandomEffects(fr)$Sigma.b
```

gvagpmim

Gaussian Variational Approximation to Gaussian Process Multiple Index Model

Description

Fits the Bayesian kernel machine multiple index model using Gaussian variational approximation algorithm.

Usage

```
gvagpmim(
  y,
  X,
  Z,
```

```

m.index = NULL,
id = NULL,
random.slope = NULL,
priors = list(),
covstr = c("diagonal", "fullrank", "blockdiag", "sparseprec"),
control = list(),
minibatch = TRUE,
verbose = TRUE,
seed = sample.int(.Machine$integer.max, 1)
)

```

Arguments

<code>y</code>	a vector of response of length <code>n</code> .
<code>X</code>	a matrix of covariates for parametric term. Should not contain an intercept.
<code>Z</code>	a matrix or a list of predictor variables to be included in nonparametric part.
<code>m.index</code>	a list of column indices grouping the predictor variables in nonparametric part. If <code>Z</code> is a matrix, <code>m.index</code> must be given.
<code>id</code>	optional vector (of length <code>n</code>) of grouping factors for fitting a model with random effects (including both a random intercept and a random slope). If <code>NULL</code> then no random effects will be included.
<code>random.slope</code>	a column index of the matrix (<code>X</code>) including covariates for random slope. If <code>NULL</code> and <code>id</code> is given, the model considers the random intercept only.
<code>priors</code>	a list giving the prior information. The list includes the following parameters (with default values in parentheses): <code>asig</code> (0.001) and <code>bsig</code> (0.001) giving the hyper parameters for σ^2 , <code>alam</code> (0.1) and <code>blam</code> (0.01) giving the hyper parameters for λ_f , <code>lam0</code> (1) and <code>tau0</code> (1) giving the hyper parameters of the horseshoe prior. The element <code>lengthscale</code> selects the prior on the reparameterized index coefficients and is one of 'indep.normal' (independent normal priors, the default) or 'horseshoe' (a continuous shrinkage prior, which performs variable selection). There is no isotropic option here, since the index parameterization already carries one component per exposure. 'normal' is accepted as a deprecated alias for 'indep.normal'.
<code>covstr</code>	Either "diagonal" (the default), "fullrank", "blockdiag", or "sparseprec", indicating which covariance structure of variational distribution is used. The "diagonal" option uses a fully factorized Gaussian for the approximation, and is available for every model. For a model without random effects, the "fullrank" option uses a Gaussian with an unrestricted lower triangular Cholesky factor. For the mixed effect models the parameter vector is partitioned by cluster and an unrestricted factor is not offered: the "blockdiag" option uses a Gaussian whose covariance matrix is block diagonal, with one block per cluster and one block for the parameters shared by all clusters, and the "sparseprec" option uses a Gaussian with a sparse precision matrix, which adds the dependence between each cluster and the shared parameters. An option that does not apply to the model at hand falls back to the closest one that does, with a warning: "fullrank" becomes "blockdiag" when random effects are present, and "blockdiag" and "sparseprec" become "fullrank" when they are not.
<code>control</code>	a named list of parameters to control the algorithm's behavior. The list includes the following parameters (with default values in parentheses): <code>max_iter</code> (100000) giving the maximum number of iterations, <code>rho</code> (0.95) giving the decaying constant, <code>eps</code> (1e-6) giving the small positive constant added to ensure

the denominator of the step size is positive and the initial step size is nonzero, nws (2500) giving rolling window size for calculating the moving average of the lower bounds, batch.size (100) giving the size of each mini-batch when minibatch = TRUE, or equivalently nbatch giving their number (supplying both is an error unless they agree), and nsp (100) giving the maximum patience parameter. The loop stops once the rolling mean of the lower bound has failed to improve on its best value for nsp consecutive updates, and the message printed on stopping – "The algorithm has converged" – reports that this counter was reached, not that the bound has settled. A small nsp can therefore stop on a run of unlucky updates and still report success. Read the plateau in plot() before trusting a fit that stopped early, and raise nsp if the bound is still climbing.

minibatch	TRUE or FALSE (default TRUE): if TRUE the data are split into batches of control\$batch.size and max_iter counts epochs rather than updates. A split that would give fewer than two batches is not a mini-batch, so the full data are used instead and a warning is given; with the default batch size of 100 that is every sample smaller than 200. The option does not apply to the random effects models.
verbose	TRUE or FALSE: flag indicating whether to print intermediate diagnostic information during the model fitting.
seed	The seed for random number generation. The default is generated from 1 to the maximum integer supported by R on the machine.

Details

Jo et al. (2026) proposed the Bayesian semiparametric regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_i = x_i^\top \beta + f^M(z_{i1}^\top \alpha_1, \dots, z_{iM}^\top \alpha_M) + \epsilon_i, \quad \epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2),$$

$$f_i^M = f^M(z_{i1}^\top \alpha_1, \dots, z_{iM}^\top \alpha_M),$$

$$f^M = (f_1^M, \dots, f_D^M)^\top \sim GP(0, \sigma^2 \lambda_f K_D),$$

where K_D denotes the RBF kernel given as

Varying lengthscale parameters:

$$K_D = \left(\exp \left(- \sum_{m=1}^M ((z_{im} - z_{jm})^\top D_m^{-1} \gamma_m)^2 \right) \right)_{i,j=1}^D, \quad \gamma_m = (\gamma_{m1}, \dots, \gamma_{mL_m})^\top,$$

where D_m is a non-singular matrix for constraints.

Jo et al. (2026) also proposed the Bayesian semiparametric mixed effects regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_{ir} = x_{ir}^\top \beta + f^M(z_{ir1}^\top \alpha_1, \dots, z_{irM}^\top \alpha_M) + u_{ir}^\top b_i + \epsilon_{ir}, \quad \epsilon_{ir} \stackrel{iid}{\sim} N(0, \sigma^2),$$

$$b_i = (b_{i1}, \dots, b_{iq})^\top \stackrel{iid}{\sim} N(0, \Sigma_b).$$

For the parameters, the following priors are used:

$$\pi(\beta) \propto 1,$$

$$\pi(\sigma^{-2}) = \text{Gamma}(a_\sigma, b_\sigma),$$

$$\pi(\lambda_f) = \text{Gamma}(a_\lambda, b_\lambda),$$

1) Independent Normal priors:

$$\pi(\gamma_{ml}) = N(0, \tau_0^2), \quad m = 1, \dots, M, \quad l = 1, \dots, L_m - 1$$

$$\pi(\gamma_{mL_m}) = N_+(0, \tau_0^2), \quad m = 1, \dots, M$$

2) Horseshoe prior:

$$\pi(\gamma_{ml} \mid \lambda_{ml}, \tau_m, \tau_\gamma) = N(0, \lambda_{ml}^2 \tau_m^2 \tau_\gamma^2), \quad m = 1, \dots, M, \quad l = 1, \dots, L_m - 1$$

$$\pi(\gamma_{mL_m} \mid \lambda_{mL_m}, \tau_m, \tau_\gamma) = N_+(0, \lambda_{mL_m}^2 \tau_m^2 \tau_\gamma^2), \quad m = 1, \dots, M$$

$$\pi(\lambda_{ml}) = C_+(0, \lambda_0), \quad m = 1, \dots, M, \quad l = 1, \dots, L_m,$$

$$\pi(\tau_m) = C_+(0, \tau_0), \quad m = 1, \dots, M,$$

$$\pi(\tau_\gamma) = C_+(0, \tau_{\gamma,0}),$$

where $a_\sigma, b_\sigma, a_\lambda, b_\lambda, \lambda_0, \tau_0$ and $\tau_{\gamma,0}$ are positive constants specified by users.

For more details, see Jo et al. (2026).

Value

an object of class "gpmim", which has the associated methods:

- * extractELBO
- * fitted (i.e., fitted.gpmim)
- * summary (i.e., summary.gpmim)
- * predict (i.e., predict.gpmim)
- * plot (i.e., plot.gpmim)

Author(s)

Seongil Jo and Woojoo Lee

References

- Jo, S., Ye, S., Hanh, G., and Lee, W. (2026), "Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior", *Statistics and Computing*, 36:79.
- Titsias, M. K. and Lazaro-Gredilla, M. (2014), "Doubly stochastic variational Bayes for non-conjugate inference", *Proceedings of the 31st ICML*.
- Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.
- Chen, H., Zheng, L., Kontai, R. A., and Raskutti, G. (2022), "Gaussian process parameter estimation using mini-batch stochastic gradient descent: convergence guarantees and empirical benefits", *Journal of Machine Learning Research*, 23, 1-59.
- McGee, G., Wilson, A., Webster, T. F., Coull, B. A. (2021), "Bayesian multiple index models for environmental mixtures", *Biometrics*, 1-13.

See Also

extractELBO, fitted.gpmim, predict.gpmim, plot.gpmim, summary.gpmim

Examples

```
## five exposures in three indices; the first index carries the surface, the
## second a weaker one, the third nothing
set.seed(1)
n <- 100
Z <- matrix(runif(n * 5, -1, 1), n, 5,
            dimnames = list(NULL, c("As", "Cd", "p1", "p2", "s1")))
X <- cbind(x = rnorm(n))
m.index <- list(metals = 1:2, phenols = 3:4, single = 5)
s1 <- as.numeric(Z[, 1:2] %*% c(0.8, 0.6))
s2 <- as.numeric(Z[, 3:4] %*% c(0.7, 0.7))
h <- (2 * s1 + 0.5)^2 - s2^2
y <- 0.5 * X[, 1] + h - mean(h) + rnorm(n, sd = 0.5)

## the horseshoe prior on the index coordinates gives inclusion probabilities
fout <- gvagpmim(y, X, Z, m.index, priors = list(lengthscale = "horseshoe"),
                minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fout, nsamples = 300) # 'alpha.' rows: index weights; 'gamma.': length-scales
plot(fout)
extractELBO(fout, nsamples = 100)
computePPIPs(fout, nsamples = 200)
```

gvagpr

Gaussian Variational Approximation to Gaussian Process Regression

Description

Fits the Bayesian kernel machine regression using Gaussian variational approximation algorithm.

Usage

```
gvagpr(
  y,
  X,
  Z,
  id = NULL,
  random.slope = NULL,
  priors = list(),
  covstr = c("diagonal", "fullrank", "blockdiag", "sparseprec"),
  inits = list(),
  control = list(),
  minibatch = TRUE,
  verbose = TRUE,
  seed = sample.int(.Machine$integer.max, 1)
)
```

Arguments

y	a vector of response of length n.
X	an n-by-p matrix of covariates for parametric term. Should not contain an intercept.

<code>Z</code>	an n-by-M matrix of predictor variables to be included in nonparametric part.
<code>id</code>	optional vector (of length n) of grouping factors for fitting a model with random effects (including both a random intercept and a random slope). If NULL then no random effects will be included.
<code>random.slope</code>	a column index of the matrix (X) including covariates for random slope. If NULL and <code>id</code> is given, the model considers the random intercept only.
<code>priors</code>	a list giving the prior information. The list includes the following parameters (with default values in parentheses): <code>asig</code> (0.001) and <code>bsig</code> (0.001) giving the hyper parameters for σ^2 , <code>alam</code> (0.1) and <code>blam</code> (0.01) giving the hyper parameters for λ_f , <code>lam0</code> (1) and <code>tau0</code> (1) giving the hyper parameters of the horseshoe prior. The element <code>lengthscale</code> selects the prior on the inverse length-scale parameters and is one of 'normal' (a single length-scale shared by all exposures), 'indep.normal' (one length-scale per exposure, the default when <code>ncol(Z) > 1</code>) or 'horseshoe' (one per exposure with a continuous shrinkage prior, which performs variable selection).
<code>covstr</code>	Either "diagonal" (the default), "fullrank", "blockdiag", or "sparseprec", indicating which covariance structure of variational distribution is used. The "diagonal" option uses a fully factorized Gaussian for the approximation, and is available for every model. For a model without random effects, the "fullrank" option uses a Gaussian with an unrestricted lower triangular Cholesky factor. For the mixed effect models the parameter vector is partitioned by cluster and an unrestricted factor is not offered: the "blockdiag" option uses a Gaussian whose covariance matrix is block diagonal, with one block per cluster and one block for the parameters shared by all clusters, and the "sparseprec" option uses a Gaussian with a sparse precision matrix, which adds the dependence between each cluster and the shared parameters. An option that does not apply to the model at hand falls back to the closest one that does, with a warning: "fullrank" becomes "blockdiag" when random effects are present, and "blockdiag" and "sparseprec" become "fullrank" when they are not.
<code>inits</code>	a named list of starting values for the variational means, given by name rather than by position. Every element is on the natural scale and is logged internally where the parameter is positive; a scalar is recycled over a block. The names are <code>beta</code> (the regression coefficients), <code>lengthscale</code> (the inverse length-scales γ), <code>lambda</code> (the scale of the Gaussian process), <code>sigsq.eps</code> (the error variance), and, for <code>lengthscale = "horseshoe"</code> , <code>tau</code> and <code>taur</code> (the local and global shrinkage scales). Anything not named starts at zero on the working scale. The random-effect covariance is set internally and is not exposed.
<code>control</code>	a named list of parameters to control the algorithm's behavior. The list includes the following parameters (with default values in parentheses): <code>max_iter</code> (100000) giving the maximum number of iterations, <code>rho</code> (0.95) giving the decaying constant, <code>eps</code> (1e-6) giving the small positive constant added to ensure the denominator of the step size is positive and the initial step size is nonzero, <code>nws</code> (2500) giving rolling window size for calculating the moving average of the lower bounds, <code>batch.size</code> (100) giving the size of each mini-batch when <code>minibatch = TRUE</code> , or equivalently <code>nbatch</code> giving their number (supplying both is an error unless they agree), and <code>nsp</code> (100) giving the maximum patience parameter. The loop stops once the rolling mean of the lower bound has failed to improve on its best value for <code>nsp</code> consecutive updates, and the message printed on stopping – "The algorithm has converged" – reports that this counter was reached, not that the bound has settled. A small <code>nsp</code> can therefore stop on a run

	of unlucky updates and still report success. Read the plateau in <code>plot()</code> before trusting a fit that stopped early, and raise <code>nsp</code> if the bound is still climbing.
<code>minibatch</code>	TRUE or FALSE (default TRUE): if TRUE the data are split into batches of <code>control\$batch.size</code> and <code>max_iter</code> counts epochs rather than updates. A split that would give fewer than two batches is not a mini-batch, so the full data are used instead and a warning is given; with the default batch size of 100 that is every sample smaller than 200. The option does not apply to the random effects models.
<code>verbose</code>	TRUE or FALSE: flag indicating whether to print intermediate diagnostic information during the model fitting.
<code>seed</code>	The seed for random number generation. The default is generated from 1 to the maximum integer supported by R on the machine.

Details

Jo et al. (2026) proposed the Bayesian semiparametric regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_i = x_i^\top \beta + f(z_i) + \epsilon_i, \quad \epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2),$$

$$f = (f(z_1), \dots, f(z_D))^\top \sim GP(0, \sigma^2 \lambda_f K_D), \quad z_i = (z_{i1}, \dots, z_{iM})^\top,$$

where K_D denotes the RBF kernel given as

1) Equal lengthscale parameter:

$$K_D = \left(\exp \left(-\gamma \sum_{m=1}^M \|z_i - z_j\|^2 \right) \right)_{i,j=1}^D$$

2) Varying lengthscale parameters:

$$K_D = \left(\exp \left(-\sum_{m=1}^M \gamma_m \|z_{im} - z_{jm}\|^2 \right) \right)_{i,j=1}^D$$

Jo et al. (2026) also proposed the Bayesian semiparametric mixed effects regression model with Gaussian process prior based on the Radial basis function (RBF) kernel:

$$y_{ir} = x_{ir}^\top \beta + f(z_{ir}) + u_{ir}^\top b_i + \epsilon_{ir}, \quad \epsilon_{ir} \stackrel{iid}{\sim} N(0, \sigma^2),$$

$$b_i = (b_{i1}, \dots, b_{iq})^\top \stackrel{iid}{\sim} N(0, \Sigma_b).$$

For the parameters, the following priors are used:

$$\pi(\beta) \propto 1,$$

$$\pi(\sigma^{-2}) = \text{Gamma}(a_\sigma, b_\sigma),$$

$$\pi(\lambda_f) = \text{Gamma}(a_\lambda, b_\lambda),$$

1) Normal prior:

$$\pi(\gamma) = N_+(0, \tau_0^2)$$

2) Independent Normal priors:

$$\pi(\gamma_m) = N_+(0, \tau_0^2), \quad m = 1, \dots, M$$

3) Horseshoe prior:

$$\pi(\gamma_m \mid \lambda_m, \tau_\gamma) = N_+(0, \lambda_m^2 \tau_\gamma^2), \quad m = 1, \dots, M$$

$$\pi(\lambda_m) = C_+(0, \lambda_0), \quad m = 1, \dots, M$$

$$\pi(\tau_\gamma) = C_+(0, \tau_0),$$

where $a_\sigma, b_\sigma, a_\lambda, b_\lambda, \lambda_0$ and τ_0 are positive constants specified by users.

For more details, see Jo et al. (2026).

Value

an object of class "gpr", which has the associated methods:

- * extractELBO
- * fitted (i.e., fitted.gpr)
- * summary (i.e., summary.gpr)
- * predict (i.e., predict.gpr)
- * plot (i.e., plot.gpr)
- * extractRandomEffects, for a fit with random effects

For a fit with random effects the rows of y, X, Z and U in the object are sorted by cluster, as the optimiser requires; order is the permutation that was applied and id.labels the cluster labels in the order of the integer codes in id. fitted returns its values in the order the data were supplied.

Author(s)

Seongil Jo and Woojoo Lee

References

- Jo, S., Ye, S., Hanh, G., and Lee, W. (2026), "Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior", *Statistics and Computing*, 36:79.
- Titsias, M. K. and Lazaro-Gredilla, M. (2014), "Doubly stochastic variational Bayes for non-conjugate inference", *Proceedings of the 31st ICML*.
- Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.
- Tan, L. S. L. and Nott D. J. (2018), "Gaussian variational approximation with sparse precision matrices", *Statistics and Computing*, 28, 259-275.
- Chen, H., Zheng, L., Kontai, R. A., and Raskutti, G. (2022), "Gaussian process parameter estimation using mini-batch stochastic gradient descent: convergence guarantees and empirical benefits", *Journal of Machine Learning Research*, 23, 1-59.
- Tan, L. S. L. and Nott, D. J. (2018), "Gaussian variational approximation with sparse precision matrices", *Statistics and Computing*, 28, 259-275.

See Also

extractELBO, fitted.gpr, predict.gpr, plot.gpr, summary.gpr

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
summary(fout)
plot(fout) # the smoothed bound along the optimisation
extractELBO(fout, nsamples = 200)
fitted(fout, cov = FALSE)$fmean[1:5]
predict(fout, Z_new = Z[1:3, ], cov = FALSE)$fmean

## a random intercept per cluster
id <- rep(1:12, each = 10)
yr <- y + rnorm(12, sd = 0.5)[id]
fr <- gvagpr(yr, X, Z, id = id, covstr = "blockdiag", verbose = FALSE, seed = 1)
extractRandomEffects(fr)$Sigma.b
```

metals

Urinary metal concentrations and hypertension in NHANES

Description

Adults aged 20 and over from four cycles of the National Health and Nutrition Examination Survey (2011–2012 through 2017–2018) who were part of the urinary metals subsample and had complete records on the variables below. Concentrations are as reported by the survey and are not transformed; the analysis in the package vignette and in Jo, Ye, Hahn and Lee (2026) takes logarithms and standardizes them.

Usage

```
metals
```

Format

A data frame with 5809 rows and 23 columns.

Ba, Cd, Co, Cs, Mo, Mn, Pb, Sb, Sn, Tl, W urinary concentrations of barium, cadmium, cobalt, caesium, molybdenum, manganese, lead, antimony, tin, thallium and tungsten, in ug/L.

sbp, dbp systolic and diastolic blood pressure in mmHg, averaged over the available readings.

meds logical: reported use of antihypertensive medication.

htn hypertension, defined as sbp ≥ 140 , dbp ≥ 90 or meds. Redefine it from sbp, dbp and meds to use another threshold.

age age in years.

sex factor, "male" or "female".

race factor with levels "MexAm", "OthHisp", "NHWhite", "NHBlack", "NHAsian" and "Other".

pir ratio of family income to the poverty threshold.

bmi body mass index in kg/m².

ucr urinary creatinine in mg/dL.

cotinine serum cotinine in ng/mL.

cycle factor giving the survey cycle.

Source

National Health and Nutrition Examination Survey, cycles 2011–2012, 2013–2014, 2015–2016 and 2017–2018. <https://www.cdc.gov/nchs/nhanes/>

References

Jo S, Ye S, Hahn G, Lee W (2026). Accelerated Bayesian Kernel Machine Regression: A Gaussian Variational Approximation with the Horseshoe Prior. *Statistics and Computing*, **36**, 79.

Examples

```
data(metals)
table(metals$htn)
summary(metals$Pb)
```

negbinomial	<i>Negative binomial family for count outcomes</i>
-------------	--

Description

A family object for the negative binomial distribution with the log link, for use with `gvaggpr` and `gvaggpim`. The linear predictor is the log mean, as in `stats::glm` and `MASS::glm.nb`, so that `exp(beta)` is a rate ratio.

Usage

```
negbinomial(link = "log")
```

Arguments

link the link function. Only "log" is available.

Details

The dispersion is always estimated: overdispersion is the reason to use a negative binomial model in the first place, so there is no option to hold it fixed, and the family therefore takes no dispersion argument. A starting value may be supplied through `inits$theta` in the call to `gvaggpr` or `gvaggpim`; see the examples. The scale of the half-normal prior on $1/\sqrt{r}$ is `priors$s_theta` in both.

`MASS::negative.binomial(theta = )` is also accepted, with its `theta` taken as a starting value rather than as a fixed dispersion, and a warning to that effect.

Value

an object of class "family".

See Also

[gvaggpr](#), [gvaggpmm](#)

Examples

```
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3)
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- rnbino(n, size = 2, mu = exp(1 + 0.3 * h))

fit <- gvaggpr(y, X, Z, family = negbinomial(), control = list(ninpts = 30),
               minibatch = FALSE, verbose = FALSE, seed = 1)
fit$theta.hat      # the estimated dispersion (true value 2)

## with a starting value for the dispersion
fit2 <- gvaggpr(y, X, Z, family = negbinomial(), inits = list(theta = 5),
                control = list(ninpts = 30), minibatch = FALSE, verbose = FALSE,
                seed = 1)

## a count outcome in the multiple index model (the iteration caps keep the
## example short; drop them for a real analysis)
m.index <- list(1:2, 3)
fitm <- gvaggpmm(y, X, Z, m.index, family = negbinomial(),
                 control = list(ninpts = 20, max_iter = 2000, nws = 500, nsp = 30),
                 minibatch = FALSE, verbose = FALSE, seed = 1)

summary(fitm)
```

plot.gpmim

Plot Diagnostics for a gpmim Object

Description

Provides a plot of the smoothed evidence lower bound (ELBO) against iterations for checking the convergence.

Usage

```
## S3 method for class 'gpmim'
plot(x, from = 1, ...)
```

Arguments

x	gpmim object, result of gvaggpmm .
from	(positive integer) first iteration to show. The default, 1, plots the whole trace.
...	unused argument.

Details

The bound rises steeply over the first few iterations, often by several orders of magnitude, and on the full range that early climb takes up the whole vertical axis: the plateau where convergence is actually decided is then flat to within a pixel. Use `from` to start the plot after the climb and read the plateau.

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpmim

plot.gpr

Plot Diagnostics for a gpr Object

Description

Provides a plot of the smoothed evidence lower bound (ELBO) against iterations for checking the convergence.

Usage

```
## S3 method for class 'gpr'
plot(x, from = 1, ...)
```

Arguments

<code>x</code>	gpr object, result of gvagpr or gvaggpr .
<code>from</code>	(positive integer) first iteration to show. The default, 1, plots the whole trace.
<code>...</code>	unused argument.

Details

The bound rises steeply over the first few iterations, often by several orders of magnitude, and on the full range that early climb takes up the whole vertical axis: the plateau where convergence is actually decided is then flat to within a pixel. Use `from` to start the plot after the climb and read the plateau.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr

plot.gprfitBivar	<i>Plot bivariate predictor-response function on a new grid of points</i>
------------------	---

Description

Provides a plot of bivariate predictor-response function on a new grid of points

Usage

```
## S3 method for class 'gprfitBivar'
plot(x, ...)
```

Arguments

x	gpr object, result of predictorResponseBivar .
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr predictorResponseBivar

plot.gprfitBivarLevels	<i>Plot interactions</i>
------------------------	--------------------------

Description

Provides a plot of the predictor-response function of a single predictor in Z for the second predictor in Z fixed at various quantiles

Usage

```
## S3 method for class 'gprfitBivarLevels'
plot(x, ...)
```

Arguments

x	gpr object, result of predictorResponseBivarLevels .
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr predictorResponseBivar

plot.gprfitUnivar	<i>Plot univariate predictor-response function on a new grid of points</i>
-------------------	--

Description

Provides a plot of univariate predictor-response function on a new grid of points

Usage

```
## S3 method for class 'gprfitUnivar'
plot(x, ...)
```

Arguments

x	gpr object, result of predictorResponseUnivar .
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr predictorResponseUnivar

plot.risks	<i>Plot Risk Summaries</i>
------------	----------------------------

Description

Provides a plot of risk summaries

Usage

```
## S3 method for class 'risks'
plot(x, ...)
```

Arguments

x	risks object, result of computeOverallRisk, computeSingVarRisk, or computeSingVarInt
...	unused argument

Author(s)

Seongil Jo

See Also

gvagpr computeOverallRisk computeSingVarRisk computeSingVarInt

predict.gpmim	<i>Extract GPMIM Predicted Values</i>
---------------	---------------------------------------

Description

predicted is a generic function which extracts predicted values for nonparametric part from an object of class "gpmim"

Usage

```
## S3 method for class 'gpmim'
predict(
  object,
  Z_new,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

object	an object of class gpmim.
Z_new	a matrix of new predictor values at which to predict new f , where each row represents a new observation.
nsamples	(positive integer) number of draws from the variational posterior. Used only when <code>variance = "total"</code> ; the predictive surface itself needs no draws. Default value is 1000.
cov	logical; if TRUE (the default) the covariance of the new f is returned as well. With many new points this is the dominant cost, so set it to FALSE when only the predictive mean is wanted.
variance	one of "conditional" (the default) or "total", choosing which covariance <code>fcov</code> reports. See the Value section.
...	additional arguments affecting the predictions produced.

Value

fmean	the exposure–response surface at the new points, evaluated at the posterior median of the parameters, $E[f \mid y, \hat{\theta}]$. This is a plug-in and not the posterior mean $E[f \mid y]$.
fcov	NULL if <code>cov = FALSE</code> . With <code>variance = "conditional"</code> , the covariance of the new f at that same $\hat{\theta}$. It drops $\text{Var}_{\theta}(E[f \mid y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$, so it is usually the narrower of the two but not necessarily at every point. With <code>variance = "total"</code> , the posterior covariance itself, by the law of total variance over <code>nsamples</code> draws; that costs one factorisation per draw and is much slower.

an object of class "gprfit", which has the associated method:

* plot (i.e., plot.gprfit)

Author(s)

Seongil Jo

See Also

gvagpmim, gvaggpmim

predict.gpr

*Extract GPR and GGPR Model Predicted Values***Description**

predicted is a generic function which extracts predicted values for nonparametric part from an object of class "gpr"

Usage

```
## S3 method for class 'gpr'
predict(
  object,
  Z_new,
  nsamples = 1000,
  cov = TRUE,
  variance = c("conditional", "total"),
  ...
)
```

Arguments

object	an object of class gpr.
Z_new	a matrix of new predictor values at which to predict new f , where each row represents a new observation.
nsamples	(positive integer) number of draws from the variational posterior. Used only when <code>variance = "total"</code> ; the predictive surface itself needs no draws. Default value is 1000.
cov	logical; if TRUE (the default) the covariance of the new f is returned as well. With many new points this is the dominant cost, so set it to FALSE when only the predictive mean is wanted.
variance	one of "conditional" (the default) or "total", choosing which covariance <code>fcov</code> reports. See the Value section.
...	additional arguments affecting the predictions produced.

Value

fmean	the exposure–response surface at the new points, evaluated at the posterior median of the parameters, $E[f \mid y, \hat{\theta}]$. This is a plug-in and not the posterior mean $E[f \mid y]$.
-------	--

`fcov` NULL if `cov = FALSE`. With `variance = "conditional"`, the covariance of the new f at that same $\hat{\theta}$. It drops $\text{Var}_{\theta}(E[f \mid y, \theta])$ and evaluates what is left at a single θ rather than averaging it over $q(\theta)$, so it is usually the narrower of the two but not necessarily at every point. With `variance = "total"`, the posterior covariance itself, by the law of total variance over n samples draws; that costs one factorisation per draw and is much slower.

an object of class "gprfit", which has the associated method:

* `plot` (i.e., `plot.gprfit`)

Author(s)

Seongil Jo

See Also

`gvagpr`, `gvaggpr`

`predictorResponseBivar`

Predict the exposure-response function at a new grid of points

Description

Predict the exposure-response function at a new grid of points

Usage

```
predictorResponseBivar(
  fit,
  z.pairs = NULL,
  ngrid = 50,
  q.fixed = 0.5,
  nsamples = 1000,
  min.plot.dist = 0.5,
  center = TRUE,
  verbose = TRUE,
  ...
)
```

Arguments

<code>fit</code>	an object of class <code>gpr</code>
<code>z.pairs</code>	data frame showing which pairs of predictors to plot
<code>ngrid</code>	number of grid points in each dimension
<code>q.fixed</code>	a second quantile at which to compare the estimated f function
<code>nsamples</code>	(positive integer) number of posterior samples. Default value is 1000
<code>min.plot.dist</code>	specifies a minimum distance that a new grid point needs to be from an observed data point in order to compute the prediction; points further than this will not be computed

center	flag for whether to scale the exposure-response function to have mean zero
verbose	TRUE or FALSE: flag of whether to print intermediate output to the screen
...	additional arguments affecting the predictions produced

Value

a long data frame with the name of the first predictor, the name of the second predictor, the value of the first predictor, the value of the second predictor, the posterior mean estimate, and the posterior standard deviation of the estimated exposure response function

References

Bobb J (2023). `_bkmr: Bayesian Kernel Machine Regression_`. R package version 0.2.2, <<https://github.com/jenfb/bkmr>>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
## every pair of exposures on a 10-by-10 grid
pred.resp.bivar <- predictorResponseBivar(fit = fout, min.plot.dist = 1, ngrid = 10,
                                         nsamples = 300, verbose = FALSE)

head(pred.resp.bivar)
```

predictorResponseBivarLevels

Plot cross-sections of the bivariate predictor-response function

Description

Function to plot the f function of a particular variable at different levels (quantiles) of a second variable

Usage

```
predictorResponseBivarLevels(
  object,
  Z = NULL,
  qs = c(0.25, 0.5, 0.75),
  both_pairs = TRUE
)
```

Arguments

object	object obtained from running the function <code>predictorResponseBivar</code>
Z	an n-by-M matrix of predictor variables to be included in nonparametric part.
qs	vector of quantiles at which to fix the second variable
both_pairs	flag indicating whether, if $h(z_1)$ is being plotted for z_2 fixed at different levels, that they should be plotted in the reverse order as well (for $h(z_2)$ at different levels of z_1)

Value

a long data frame with the name of the first predictor, the name of the second predictor, the value of the first predictor, the quantile at which the second predictor is fixed, the posterior mean estimate, and the posterior standard deviation of the estimated exposure response function

References

Bobb J (2023). `_bkmr: Bayesian Kernel Machine Regression_`. R package version 0.2.2, <<https://github.com/jenfb/bkmr>>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
## every pair of exposures on a 10-by-10 grid
pred.resp.bivar <- predictorResponseBivar(fit = fout, min.plot.dist = 1, ngrid = 10,
                                         nsamples = 300, verbose = FALSE)

head(pred.resp.bivar)
## the surfaces sliced at quantiles of the second exposure of each pair
pred.resp.bivar.levels <- predictorResponseBivarLevels(pred.resp.bivar, Z = Z,
                                                       qs = c(0.1, 0.5, 0.9))

plot(pred.resp.bivar.levels)
```

`predictorResponseBivarPair`

Predict bivariate predictor-response function on a new grid of points

Description

Predict bivariate predictor-response function on a new grid of points

Usage

```
predictorResponseBivarPair(
  fit,
  whichz1 = 1,
  whichz2 = 2,
  whichz3 = NULL,
  prob = 0.5,
  q.fixed = 0.5,
  nsamples = 1000,
  ngrid = 50,
  min.plot.dist = 0.5,
  center = TRUE,
  ...
)
```

Arguments

<code>fit</code>	an object of class <code>gpr</code>
<code>whichz1</code>	vector identifying the first predictor that (column of Z) should be plotted
<code>whichz2</code>	vector identifying the second predictor that (column of Z) should be plotted
<code>whichz3</code>	vector identifying the third predictor that will be set to a pre-specified fixed quantile (determined by <code>prob</code>)
<code>prob</code>	pre-specified quantile to set the third predictor (determined by <code>whichz3</code>); defaults to 0.5 (50th percentile)
<code>q.fixed</code>	a second quantile at which to compare the estimated <code>f</code> function
<code>nsamples</code>	(positive integer) number of posterior samples. Default value is 1000
<code>ngrid</code>	number of grid points in each dimension
<code>min.plot.dist</code>	specifies a minimum distance that a new grid point needs to be from an observed data point in order to compute the prediction; points further than this will not be computed
<code>center</code>	flag for whether to scale the exposure-response function to have mean zero
<code>...</code>	additional arguments affecting the predictions produced

Value

a data frame with value of the first predictor, the value of the second predictor, the posterior mean estimate, and the posterior standard deviation

References

Bobb J (2023). `_bkmr: Bayesian Kernel Machine Regression_`. R package version 0.2.2, <<https://github.com/jenfb/bkmr>>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
```

```

h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
## a 10-by-10 grid keeps the example quick
pred.resp.bivar12 <- predictorResponseBivarPair(fit = fout, min.plot.dist = 1,
                                                ngrid = 10, nsamples = 300)

head(pred.resp.bivar12)

```

predictorResponseUnivar

Predict univariate predictor-response function on a new grid of points

Description

Predict univariate predictor-response function on a new grid of points

Usage

```

predictorResponseUnivar(
  fit,
  which.z = 1:ncol(Z),
  ngrid = 50,
  q.fixed = 0.5,
  nsamples = 1000,
  min.plot.dist = Inf,
  center = TRUE,
  z.names = colnames(Z),
  ...
)

```

Arguments

fit	an object of class gpr
which.z	vector identifying which predictors (columns of Z) should be plotted
ngrid	number of grid points to cover the range of each predictor (column in Z)
q.fixed	a second quantile at which to compare the estimated f function
nsamples	(positive integer) number of posterior samples. Default value is 1000
min.plot.dist	specifies a minimum distance that a new grid point needs to be from an observed data point in order to compute the prediction; points further than this will not be computed
center	flag for whether to scale the exposure-response function to have mean zero
z.names	a vector of names of predictors Z. Default values are colnames(Z).
...	additional arguments affecting the predictions produced.

Value

a long data frame with the predictor name, predictor value, posterior mean estimate, and posterior standard deviation

References

Bobb, J. F., Valeri, L., Claus, H. B., Christiani, D. C., Wright, R. O., Mazumdar, M., Godleski, J. J., and Coull, B. A. (2015). "Bayesian Kernel Machine Regression for Estimating the Health Effects of Multi-Pollutant Mixtures", *Biostatistics*, 16, 493-508.

Bobb J (2023). `_bkmr`: Bayesian Kernel Machine Regression_. R package version 0.2.2, <<https://github.com/jenfb/bkmr>>

Examples

```
## a small simulated data set: three exposures, one covariate, a surface in
## the first two exposures
set.seed(1)
n <- 120
Z <- matrix(runif(n * 3, -1, 1), n, 3, dimnames = list(NULL, c("z1", "z2", "z3")))
X <- cbind(x = rnorm(n))
h <- 2 * Z[, 1]^2 - Z[, 2]
y <- 0.5 * X[, 1] + h + rnorm(n, sd = 0.5)
fout <- gvagpr(y, X, Z, minibatch = FALSE, verbose = FALSE, seed = 1)
pred.resp.univar <- predictorResponseUnivar(fout, ngrid = 20, nsamples = 300)
plot(pred.resp.univar)
```

print.gpmim

Print basic summary of gpmim and ggpmim model fit

Description

print method for class "gpmim"

Usage

```
## S3 method for class 'gpmim'
print(x, ...)
```

Arguments

`x` an object of class gpmim.
`...` unused argument.

Author(s)

Seongil Jo

See Also

gvagpmim, gvagppmim

print.gpr	<i>Print basic summary of gpr and ggpr model fit</i>
-----------	--

Description

print method for class "gpr"

Usage

```
## S3 method for class 'gpr'
print(x, ...)
```

Arguments

x	an object of class gpr.
...	unused argument.

Author(s)

Seongil Jo

See Also

gvagpr, gvaggpr

summary.gpmim	<i>Summarizing gpmim and ggpmim model fits</i>
---------------	--

Description

summary method for class "gpmim"

Usage

```
## S3 method for class 'gpmim'
summary(
  object,
  q = c(0.025, 0.975),
  digits = 5,
  nsamples = 1000,
  ntuning = 10,
  ...
)
```

Arguments

object	an object of class gpmim
q	quantiles of posterior distribution (credible interval) to show
digits	the number of digits to show when printing
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
ntuning	the number of chosen values of the tuning parameter for variable selection, defaults to 10
...	unused argument.

Value

the table of posterior summaries of the model parameters – mean, standard deviation and the quantiles in `q` – returned invisibly; `summary` is called for what it prints. The rows are the regression coefficients, the error variance, the kernel amplitude `lambda`, and then three blocks that describe the index structure. `gamma.<index>` is the length-scale of an index, $\gamma_m = \sum_l \psi_{ml}^2$, the multiple-index counterpart of the per-exposure length-scales that `summary.gpr` reports. `psi.<exposure>` is the weight of one exposure inside its index on the same scale, signed, so that an interval covering zero says the exposure does not enter the index. `alpha.<exposure>` is that weight normalised, $\alpha_m = \psi_m / \|\psi_m\|$, which gives the composition of an index but not its size: it lies on the unit sphere, so a dead index still shows weights of order one. Pseudo posterior inclusion probabilities are printed for the shrinkage priors and are obtained on their own with `computePPIPs`.

Author(s)

Seongil Jo

See Also

`gvagpmmim`, `gvagpmmim`

summary.gpr

Summarizing gpr and ggpr model fits

Description

`summary` method for class "gpr"

Usage

```
## S3 method for class 'gpr'
summary(
  object,
  q = c(0.025, 0.975),
  digits = 5,
  nsamples = 1000,
  ntuning = 10,
  ...
)
```

Arguments

object	an object of class gpr
q	quantiles of posterior distribution (credible interval) to show
digits	the number of digits to show when printing
nsamples	(positive integer), number of posterior samples to draw and save, defaults to 1000
ntuning	the number of chosen values of the tuning parameter for variable selection, defaults to 10
...	unused argument.

Value

the table of posterior summaries of the model parameters – mean, standard deviation and the quantiles in `q` – returned invisibly; `summary` is called for what it prints. The same table is returned whichever length-scale prior was fitted, so that code downstream does not have to branch on it. Pseudo posterior inclusion probabilities are printed for the shrinkage priors and are obtained on their own with `computePPIPs`.

Author(s)

Seongil Jo

See Also

`gvagpr`, `gvaggpr`

Index

* datasets

metals, [30](#)

computeOverallRisk, [2](#)

computePPIPs, [3](#)

computeSingVarInt, [5](#)

computeSingVarRisk, [6](#)

extractELBO, [7](#), [16](#)

extractPostSamps, [8](#)

extractRandomEffects, [9](#)

fitted.gpmim, [9](#)

fitted.gpr, [10](#)

gvaggpmim, [12](#), [31](#), [32](#)

gvaggpr, [14](#), [18](#), [31](#), [32](#)

gvagpmim, [22](#)

gvagpr, [26](#)

metals, [30](#)

negbinomial, [31](#)

plot.gpmim, [32](#)

plot.gpr, [33](#)

plot.gprfitBivar, [34](#)

plot.gprfitBivarLevels, [34](#)

plot.gprfitUnivar, [35](#)

plot.risks, [35](#)

predict.gpmim, [36](#)

predict.gpr, [37](#)

predictorResponseBivar, [38](#), [40](#)

predictorResponseBivarLevels, [39](#)

predictorResponseBivarPair, [40](#)

predictorResponseUnivar, [42](#)

print.gpmim, [43](#)

print.gpr, [44](#)

summary.gpmim, [44](#)

summary.gpr, [45](#)